

# ELI — Grounding And Evidence

ELI v2.3.4

August 2026

## Contents

|                                                                                               |          |
|-----------------------------------------------------------------------------------------------|----------|
| <b>ELI Grounding &amp; Evidence Layer</b>                                                     | <b>1</b> |
| The core idea . . . . .                                                                       | 1        |
| Components . . . . .                                                                          | 2        |
| How it fits the pipeline . . . . .                                                            | 3        |
| Honest assessment . . . . .                                                                   | 3        |
| Update — 2026-06-09 . . . . .                                                                 | 3        |
| Update — 2026-06-09 (deterministic reports surfaced verbatim; examiner correctness) . . . . . | 3        |

## ELI Grounding & Evidence Layer

The anti-confabulation system — a deterministic evidence scaffold wrapped around the probabilistic model. This is ELI’s most distinctive subsystem and the part closest to genuinely frontier. Spans `eli/runtime/` and `eli/cognition/`.

### The core idea

For “grounded” actions (status, runtime, memory, identity, control), ELI does **not** trust the LLM to state facts. It (1) gathers deterministic evidence from the live system, (2) can answer directly from that evidence without the LLM at all, and (3) when the LLM does generate, validates the output against the evidence and rejects/repairs contradictions. The LLM becomes a phraser, not a source.

**Correction (2026-06-08): the “bypass” is PARTIAL and MODE-GATED — not a blanket bypass.** Earlier wording here and in `what_eli_is.md` (“bypasses the language model entirely”, “without the LLM at all”) overstated it. The running reality, verified against the engine: the deterministic verbatim return fires for a **subset** of actions and is **mode-gated** — - a small **`_verbatim_always_actions`** set (deep introspection like `EXPLAIN_MEMORY_RUNTIME`) returns verbatim in **every** reasoning mode; - **`_deterministic_direct_payload_actions`** (status/reports + router-fast/command actions: `DATE/TIME`, `VOLUME`, `WRITE_NOTE`, window/media/file ops, `NEWS/MORNING_REPORT`, self-report ...) return **verbatim in quick mode** and **synthesise in non-quick** modes (`engine._bypass_persona + _is_grounded_control_nonquick`); - everything else is the model’s to phrase.

So the model IS in the loop for most conversational turns; the “phraser, not a source” principle holds where the deterministic path actually fires, not universally. A 2026-06-08 fix corrected the membership: the command actions were in the wrong set, so quick mode re-synthesised them, corrupting results (“Wrote note”→“Bought note”) and adding latency. They are now verbatim in quick mode, synthesised in others.

## Components

### runtime/deterministic\_grounding\_gate.py (4.3k LOC)

The deterministic renderer. `render_action(action, args, user_input, mode_label)` produces a grounded answer for control/status actions directly from runtime data (settings, runtime snapshot, DB counts, GPU line) — bypassing the model. `install(CognitiveEngine)` wires it into the engine. `_eli_v14_runtime_data()` assembles the live config block (model\_path, n\_ctx, gpu\_layers, ...).

**Code-health flag:** the file contains **seven** `render_action` definitions (lines 350, 1058, 2837, 3288, 3653, 3875, 4222), each marked `# type: ignore[override]`. They are stacked successive redefinitions where the last one wins — the file grew by appending new versions rather than editing in place. It works, but it's the single clearest example of the “added beside, not folded in” pattern, and it makes the effective code path hard to trace. Prime consolidation candidate.

### runtime/control\_contracts.py (943 LOC)

The deterministic control path: - `is_control_action / route_control_text` — recognise control/status intents. - `build_control_evidence(engine, action, args, ...)` — gather the evidence packet (runtime paths, DB state, bus result, trace). - **`output_violates_evidence(text, evidence_text)`** — the gate that returns True when LLM output contradicts/omits the evidence; the engine uses this to reject a hallucinated answer. - `compact_evidence_answer / finalise_control_result` — assemble the final grounded response.

### runtime/evidence\_ledger.py (595 LOC)

A persistent SQLite ledger of evidence events: `record_event`, `recent_events`, `repeated_event_signals` (detect recurring issues over N days), `status_evidence`, `artifact_snapshot`. Gives ELI a durable, queryable record of what actually happened.

### runtime/evidence\_arbitration.py (195 LOC)

`EvidenceItem + arbitrate_evidence(limit) + build_evidence_context_text` — scores and merges competing evidence into a single context block. Pairs with the agent-bus confidence aggregation (`_score_tool_result`).

### runtime/memory\_evidence.py + runtime/retrieval\_packets.py

`collect_memory_evidence / build_memory_evidence_text` turn memory hits into an evidence block. `retrieval_packets` builds `StagePackets` for each retrieval stage (parallel-retrieval, hybrid-merge, rerank, source-trace) — provenance so the pipeline can show *where* a fact came from.

### cognition/output\_governor.py (1224 LOC)

Post-generation governor: `govern_output(text, is_grounded)`, `normalize_assistant_text`, `validate_against_evidence`, plus a family of drift-repair functions — `strip_generic_ai_identity_drift` (kills “As an AI language model...”), `repair_local_persona_drift`, `repair_self_user_confusion` (fixes the model conflating itself with the user), `clean_response_style`. The last line of defence before text reaches the user.

### cognition/grounded\_status.py (644 LOC)

`direct_grounded_answer(user_text)` — fully deterministic answers for identity / memory-inventory / status questions, assembled from the DBs (`format_user_identity`, `format_memory_inventory`, table distributions). No LLM.

## runtime/grounded\_remediation.py (1.6k LOC)

The failure→repair loop. When an action fails (app won't open, path missing, browser/IDE absent), it: diagnose\_app/path/browser/ide → build\_repair\_plan → offer\_for\_result ("want me to install X?") → handle\_confirmation (consumes yes/no) → execute\_pending\_plan. Stateful pending-repair tracking + explain\_last\_failure. This is what makes failures conversational and recoverable instead of dead ends.

## How it fits the pipeline

1. Router classifies action. Control/grounded actions enter the deterministic path.
2. build\_control\_evidence / collect\_memory\_evidence gather facts.
3. PHASE45 deterministic bypass: for many status actions render\_action / direct\_grounded\_answer answer **without** the LLM.
4. If the LLM generates, output\_violates\_evidence + validate\_against\_evidence gate it; govern\_output cleans drift.
5. grounded\_remediation handles action failures with an offer/confirm loop.
6. evidence\_ledger records the event for future repeated\_event\_signals.

## Honest assessment

- **Strong (genuinely):** very few local-LLM projects build a deterministic evidence layer at all, let alone one this thorough — direct-answer bypass, output-vs-evidence rejection, drift repair, a persistent ledger, and a conversational remediation loop. This is the subsystem that most justifies the "frontier" label.
  - **Weak:**
    1. The **seven stacked render\_action overrides** in a 4.3k file — the effective behaviour is whatever the last definition does; the earlier six are dead weight that obscure the real path. Consolidate to one.
    2. **Overlapping surfaces** — grounded\_status, control\_contracts, deterministic\_grounding\_gate, and the runtime/\*\_response / \*\_surface modules all render grounded answers with partial overlap. The boundaries between "who renders the final grounded string" are blurry.
    3. Confidence/grounding is heuristic (additive score + threshold + output\_violates\_evidence), not a formal proof — fine, but it's a gate, not a guarantee.
- 

## Update — 2026-06-09

- **Self-patch only patches what it can ground.** generate\_code\_patch extracts the target file from the error traceback; when a failure has no in-project file (e.g. an HTTP/connection error, "No commands to run"), it used to ask the model anyway, which **invented** a path (phantom api\_client.py) that then failed to apply. It now returns no\_groundable\_file and routes the failure to goal-autogenesis / self-heal instead; when a file IS grounded it is **pinned** in the prompt so the model can't drift to a different path.
- **Banter guard:** the llm\_intent resolver occasionally maps playful input to a generative action; the engine now downgrades GENERATE\_SCRIPT/PROJECT/DOCUMENT/CODE SOLVE to CHAT when the text carries no create-intent keyword (real "write me a script" is preserved).
- **Compact grounded synthesis carries a VOICE primer** (pulled live from the canonical persona) so factual/introspection answers sound like ELI without losing the EXACT-FACTS contract that pins every number/path/table/DB to the evidence.

## Update — 2026-06-09 (deterministic reports surfaced verbatim; examiner correctness)

- **Verbatim guard for deterministic grounded reports.** \_get\_chat\_response now returns the EXAMINE\_CODE tiered report and the FILE\_AUDIT file-count VERBATIM when they appear in the

evidence, and surfaces the `FIX_FILE` success event as a real outcome — instead of letting the quick chat path re-narrate them. This killed three live confabulation classes: `EXAMINE_CODE` inventing “findings” that were actually existing code comments; `FILE_AUDIT` inventing files-with-bugs that don’t exist; `FIX_FILE` narrating “I cannot fix” after it had already written the file.

- **Examiner correctness (`code_examiner.py`).** Tier-2 now drops pyflakes star-import noise (“may be undefined, or defined from star imports” — ~800 false positives on the GUI → 0). Tier-3 reviews the `WHOLE` file in line-correct windows (was only the first ~`MAX_FILE_CHARS`, so god-files were ~90% unreviewed), each source line prefixed with its `REAL` number, and rejects any finding citing a line outside the shown window (kills the “`engine.py:132` undefined ‘`ov`’” hallucination class — line 132 is `or result.get("error")`).
- **Routing:** “is there any issues with the files?” / “check the code for bugs” now route to `EXAMINE_CODE` (the real examiner), not `FILE_AUDIT`; `fix-intent` (“fix the bugs in `foo.py`”) stays `FIX_FILE`.